.SHP File API

The .SHP API uses a SHPHandle to represent an open .shp/.shx file pair. The contents of the SHPHandle are visible (see shapefile.h) but should be ignored by the application. It is intended that all information be accessed by the API functions.

Shape Types

Shapes have types associated with them. The following is a list of the different shapetypes supported by Shapefiles. At this time all shapes in a Shapefile must be of the same type (with the exception of NULL shapes).

```
  #define SHPT_NULL             0

  2D Shape Types (pre ArcView 3.x):

  #define SHPT_POINT            1       Points
  #define SHPT_ARC              3       Arcs (Polylines, possible in parts)
  #define SHPT_POLYGON          5       Polygons (possible in parts)
  #define SHPT_MULTIPOINT       8       MultiPoint (related points)

  3D Shape Types (may include "measure" values for vertices):

  #define SHPT_POINTZ           11
  #define SHPT_ARCZ             13
  #define SHPT_POLYGONZ         15
  #define SHPT_MULTIPOINTZ      18

  2D + Measure Types:

  #define SHPT_POINTM           21
  #define SHPT_ARCM             23
  #define SHPT_POLYGONM         25
  #define SHPT_MULTIPOINTM      28

  Complex (TIN-like) with Z, and Measure:

  #define SHPT_MULTIPATCH       31
```

SHPObject

An individual shape is represented by the SHPObject structure. SHPObject's created with SHPCreateObject(), SHPCreateSimpleObject(), or SHPReadObject() should be disposed of with SHPDestroyObject().

```
  typedef struct
  {
    int         nSHPType;       Shape Type (SHPT_* - see list above)

    int         nShapeId;       Shape Number (-1 is unknown/unassigned)

    int         nParts;         # of Parts (0 implies single part with no info)
    int         *panPartStart;  Start Vertex of part
    int         *panPartType;   Part Type (SHPP_RING if not SHPT_MULTIPATCH)

    int         nVertices;      Vertex list
    double      *padfX;
    double      *padfY;
    double      *padfZ;         (all zero if not provided)
    double      *padfM;         (all zero if not provided)

    double      dfXMin;         Bounds in X, Y, Z and M dimensions
    double      dfYMin;
    double      dfZMin;
    double      dfMMin;

    double      dfXMax;
    double      dfYMax;
    double      dfZMax;
    double      dfMMax;
  } SHPObject;
```

SHPOpen()

```
SHPHandle SHPOpen( const char * pszShapeFile, const char * pszAccess );

  pszShapeFile:         The name of the layer to access.  This can be the
                        name of either the .shp or the .shx file or can
                        just be the path plus the basename of the pair.

  pszAccess:            The fopen() style access string.  At this time only
                        "rb" (read-only binary) and "rb+" (read/write binary)
                        should be used.
```

The SHPOpen() function should be used to establish access to the two files for accessing vertices (.shp and .shx). Note that both files have to be in the indicated directory, and must have the expected extensions in lower case. The returned SHPHandle is passed to other access functions, and SHPClose() should be invoked to recover resources, and flush changes to disk when complete.

SHPGetInfo()

```
void SHPGetInfo( SHPHandle hSHP, int * pnEntities, int * pnShapeType,
                 double * padfMinBound, double * padfMaxBound );

  hSHP:                 The handle previously returned by SHPOpen()
                        or SHPCreate().

  pnEntities:           A pointer to an integer into which the number of
                        entities/structures should be placed.  May be NULL.

  pnShapetype:          A pointer to an integer into which the shapetype
                        of this file should be placed.  Shapefiles may contain
                        either SHPT_POINT, SHPT_ARC, SHPT_POLYGON or
                        SHPT_MULTIPOINT entities.  This may be NULL.

  padfMinBound:         The X, Y, Z and M minimum values will be placed into
                        this four entry array.  This may be NULL.

  padfMaxBound:         The X, Y, Z and M maximum values will be placed into
                        this four entry array.  This may be NULL.
```

The SHPGetInfo() function retrieves various information about shapefile as a whole. The bounds are read from the file header, and may be inaccurate if the file was improperly generated.

SHPReadObject()

```
SHPObject *SHPReadObject( SHPHandle hSHP, int iShape );

  hSHP:                 The handle previously returned by SHPOpen()
                        or SHPCreate().

  iShape:               The entity number of the shape to read.  Entity
                        numbers are between 0 and nEntities-1 (as returned
                        by SHPGetInfo()).
```

The SHPReadObject() call is used to read a single structure, or entity from the shapefile. See the definition of the SHPObject structure for detailed information on fields of a SHPObject. SHPObject's returned from SHPReadObject() should be deallocated with SHPDestroyShape(). SHPReadObject() will return NULL if an illegal iShape value is requested.

Note that the bounds placed into the SHPObject are those read from the file, and may not be correct. For points the bounds are generated from the single point since bounds aren't normally provided for point types.

Generally the shapes returned will be of the type of the file as a whole. However, any file may also contain type SHPT_NULL shapes which will have no geometry. Generally speaking applications should skip rather than preserve them, as they usually represented interactively deleted shapes.

SHPClose()

```
void    SHPClose( SHPHandle hSHP );

  hSHP:                 The handle previously returned by SHPOpen()
                        or SHPCreate().
```

The SHPClose() function will close the .shp and .shx files, and flush all outstanding header information to the files. It will also recover resources associated with the handle. After this call the hSHP handle cannot be used again.

SHPCreate()

```
SHPHandle SHPCreate( const char * pszShapeFile, int nShapeType );

  pszShapeFile:         The name of the layer to access.  This can be the
                        name of either the .shp or the .shx file or can
                        just be the path plus the basename of the pair.

  nShapeType:           The type of shapes to be stored in the newly created
                        file.  It may be either SHPT_POINT, SHPT_ARC,
                        SHPT_POLYGON or SHPT_MULTIPOINT.
```

The SHPCreate() function will create a new .shp and .shx file of the desired type.

SHPCreateSimpleObject()

```
SHPObject *
     SHPCreateSimpleObject( int nSHPType, int nVertices,
                            double *padfX, double * padfY, double *padfZ, );

  nSHPType:             The SHPT_ type of the object to be created, such
                        as SHPT_POINT, or SHPT_POLYGON.

  nVertices:            The number of vertices being passed in padfX,
                        padfY, and padfZ.

  padfX:                An array of nVertices X coordinates of the vertices
                        for this object.
```

```
  padfY:                An array of nVertices Y coordinates of the vertices
                        for this object.

  padfZ:                An array of nVertices Z coordinates of the vertices
                        for this object.  This may be NULL in which case
                        they are all assumed to be zero.
```

The SHPCreateSimpleObject() allows for the convenient creation of simple objects. This is normally used so that the SHPObject can be passed to SHPWriteObject() to write it to the file. The simple object creation API assumes an M (measure) value of zero for each vertex. For complex objects (such as polygons) it is assumed that there is only one part, and that it is of the default type (SHPP_RING).

Use the SHPCreateObject() function for more sophisticated objects. The SHPDestroyObject() function should be used to free resources associated with an object allocated with SHPCreateSimpleObject().

This function computes a bounding box for the SHPObject from the given vertices.

SHPCreateObject()

```
SHPObject *
     SHPCreateObject( int nSHPType, int iShape,
                      int nParts, int * panPartStart, int * panPartType,
                      int nVertices, double *padfX, double * padfY,
                      double *padfZ, double *padfM );

  nSHPType:             The SHPT_ type of the object to be created, such
                        as SHPT_POINT, or SHPT_POLYGON.

  iShape:               The shapeid to be recorded with this shape.

  nParts:               The number of parts for this object.  If this is
                        zero for ARC, or POLYGON type objects, a single
                        zero valued part will be created internally.

  panPartStart:         The list of zero based start vertices for the rings
                        (parts) in this object.  The first should always be
                        zero.  This may be NULL if nParts is 0.

  panPartType:          The type of each of the parts.  This is only meaningful
                        for MULTIPATCH files.  For all other cases this may
                        be NULL, and will be assumed to be SHPP_RING.

  nVertices:            The number of vertices being passed in padfX,
                        padfY, and padfZ.

  padfX:                An array of nVertices X coordinates of the vertices
                        for this object.

  padfY:                An array of nVertices Y coordinates of the vertices
                        for this object.

  padfZ:                An array of nVertices Z coordinates of the vertices
                        for this object.  This may be NULL in which case
                        they are all assumed to be zero.

  padfM:                An array of nVertices M (measure values) of the
                        vertices for this object.  This may be NULL in which
                        case they are all assumed to be zero.
```

The SHPCreateSimpleObject() allows for the creation of objects (shapes). This is normally used so that the SHPObject can be passed to SHPWriteObject() to write it to the file.

The SHPDestroyObject() function should be used to free resources associated with an object allocated with SHPCreateObject().

This function computes a bounding box for the SHPObject from the given vertices.

SHPComputeExtents()

```
void SHPComputeExtents( SHPObject * psObject );

  psObject:             An existing shape object to be updated in place.
```

This function will recompute the extents of this shape, replacing the existing values of the dfXMin, dfYMin, dfZMin, dfMMin, dfXMax, dfYMax, dfZMax, and dfMMax values based on the current set of vertices for the shape. This function is automatically called by SHPCreateObject() but if the vertices of an existing object are altered it should be called again to fix up the extents.

SHPWriteObject()

```
int SHPWriteObject( SHPHandle hSHP, int iShape, SHPObject *psObject );

  hSHP:                 The handle previously returned by SHPOpen("r+")
                        or SHPCreate().

  iShape:               The entity number of the shape to write.  A value of
                        -1 should be used for new shapes.
```

```
  psObject:             The shape to write to the file. This should have
                        been created with SHPCreateObject(), or
                        SHPCreateSimpleObject().
```

The SHPWriteObject() call is used to write a single structure, or entity to the shapefile. See the definition of the SHPObject structure for detailed information on fields of a SHPObject. The return value is the entity number of the written shape.

SHPDestroyObject()

```
void SHPDestroyObject( SHPObject *psObject );

  psObject:             The object to deallocate.
```

This function should be used to deallocate the resources associated with a SHPObject when it is no longer needed, including those created with SHPCreateSimpleObject(), SHPCreateObject() and returned from SHPReadObject().

SHPRewindObject()

```
int SHPRewindObject( SHPHandle hSHP, SHPObject *psObject );

  hSHP:                 The shapefile (not used at this time).
  psObject:             The object to deallocate.
```

This function will reverse any rings necessary in order to enforce the shapefile restrictions on the required order of inner and outer rings in the Shapefile specification. It returns TRUE if a change is made and FALSE if no change is made. Only polygon objects will be affected though any object may be passed.